UNIVERSITY COLLEGE OF SCIENCE AND TECHNOLOGY
Department of Engineering Sciences and Applied Arts · Bachelor of Computer Systems Engineering

Edu Bridge
Bridges Beyond Borders

Edu Bridge – Smart Scholarship and Student Services Management System

System Analysis & Database Design

ERD · UML diagrams · Data dictionary · SQL · Activity & state models

Prepared by	**Alaa Suleiman Abdulrahman Abu Mustafa · 220217287** **Database design & integration**
With contributions from	Lamia Fathi Hamdi Madi (220217142) — Backend development (PHP API) Nada Musa Abdul Qader Al-Asital (220217234) — Frontend development (React.js) Janeen Osama Atiya Al-Qutati (220217280) — AI recommendation, CRM & evaluation
Course supervisor	Ms. Amina Al-Salout
Academic year	2026/2027
Document version	1.0 — October 2026

Document information

This report is a team deliverable of the graduation project. The table below shows which student prepared each part, so the supervisor can see the individual work of every team member.

Table 1: *Contribution of each student*

Part of this document	Prepared by	Student ID
Database design: ERDs, normalisation, keys, data dictionary, SQL, reporting views, security, backup (§9)	Alaa Suleiman Abdulrahman Abu Mustafa	220217287
System architecture, back-end class diagram, sequence diagrams, API mapping (§3, §5.2, §6, §10)	Lamia Fathi Hamdi Madi	220217142
Use case modelling and activity diagrams, user flows (§4, §7)	Nada Musa Abdul Qader Al-Asital	220217234
Domain model and state diagrams of the workflow and CRM (§5.1, §8)	Janeen Osama Atiya Al-Qutati	220217280
Requirements analysis (§2) and review of the whole document	All team members	—

Table 2: *Document details*

Item	Value
Project	Edu Bridge – Smart Scholarship and Student Services Management System
Database	MySQL 8 / MariaDB 10.4 — `edubridge` (25 tables, 7 views)
Source files	`edubridge-api/database/schema.sql`, `seed.sql`, `views.sql`
Diagram notation	UML 2.5 (use case, class, sequence, activity, state) and crow's-foot ERD

> **How to read the diagrams**
> All diagrams were produced from text definitions (Mermaid) kept in the repository, so they can be updated together with the code.

Table of Contents

Document information 2
1. Introduction 5
1.1 Purpose of this document 5
1.2 Scope 5
1.3 System overview 5
1.4 Definitions 5
2. Requirements Analysis 6
2.1 Actors 6
2.2 Functional requirements 6
2.3 Non-functional requirements 7
3. System Architecture 8
4. Use Case Modeling 10
4.1 Use case descriptions 12
5. Class Diagrams 14
5.1 Domain model 14
5.2 Back-end design classes 14
6. Sequence Diagrams 17
6.1 Sign in 17
6.2 Upload a document and update readiness 18
6.3 Service request handled by an advisor 19
7. Activity Diagrams 20
7.1 Student registration 20
7.2 Scholarship application journey 21
8. State Diagrams 22
8.1 Application (8 stages) 22
8.2 Document 23
8.3 Service request 23
8.4 Scholarship and admission results 24
8.5 User account 25
9. Database Design 26
9.1 Design approach 26
9.2 Conceptual ERD 26
9.3 Logical ERDs by module 27
9.4 Normalisation 29
9.5 Keys, constraints and indexes 30
9.6 Data dictionary 30
9.7 SQL implementation 38
9.8 Security 40

9.9 Backup and recovery.....40
10. API and Database Mapping.....42
11. Conclusion.....43

If page numbers do not appear, right-click the table and choose "Update Field".

1. Introduction

1.1 Purpose of this document

This document presents the complete system analysis and database design of **EduBridge – Smart Scholarship and Student Services Management System**. It translates the approved project proposal into analysis models (use cases, UML class, sequence, activity and state diagrams) and into a concrete relational database implemented in **MySQL / MariaDB**. Every model in this document matches the database and REST API that are already running in the project repository (`edubridge-api`).

1.2 Scope

The analysis covers the three portals of the system — Student, Advisor (Employee) and Administrator — and the automated services that support them (matching engine, readiness score, notifications). The database design covers 25 tables, their keys and constraints, reporting views, security and backup.

1.3 System overview

EduBridge replaces the scattered WhatsApp-based workflow of an educational consultancy office with one centralised web platform. Students build a profile, upload their documents once, discover suitable scholarships and follow every step of their application. Advisors manage students, documents, service requests and applications from one CRM, and administrators manage users and view statistics.

Table 3: *Technology summary*

Item	Value
Front end	React.js 18 (Vite), React Router, react-i18next (Arabic RTL / English LTR)
Back end	PHP 8 REST API (no framework), JWT authentication
Database	MySQL 8 / MariaDB 10.4 — utf8mb4, InnoDB, 25 tables, 7 reporting views
AI	Rule-based weighted matching + Groq API (pre-trained LLM) for explanations
Tools	Git & GitHub · Postman & Swagger (OpenAPI 3) · phpMyAdmin · XAMPP

1.4 Definitions

Table 4: *Definitions*

Term	Meaning
Readiness score	Percentage (0–100) showing how complete a student's required documents are.
Match score	Weighted score (0–100) showing how well a student profile fits a scholarship.
Stage	Position of an application in the 8-stage workflow (New request → Accepted / Rejected).
CRM	Customer Relationship Management — the advisor workspace for students, notes and communication.
JWT	JSON Web Token used to authenticate API requests; one token per signed-in device.

2. Requirements Analysis

2.1 Actors

Table 5: *System actors*

Actor	Description	Main goals
Student	Applicant who wants a scholarship or a consultancy service.	Build profile, upload documents, find scholarships, request services, track applications.
Advisor / Employee	Edu Bridge staff member who follows students.	Review documents, handle requests, move applications through stages, log communication.
Administrator	Manages the platform.	Manage users and roles, scholarships, reports and settings.
Scheduler + matching engine	Automated part of the system (no human).	Compute match & readiness scores, send reminders, close expired scholarships.

2.2 Functional requirements

Table 6: *Functional requirements*

ID	Requirement	Actor	Priority
FR-01	Register a student account in five steps and confirm the email address.	Student	High
FR-02	Sign in, sign out and reset the password with a 6-digit code.	All	High
FR-03	Manage personal, academic, language and preference information.	Student	High
FR-04	Upload, replace and delete documents (PDF, JPG, PNG, DOCX ≤ 10 MB).	Student	High
FR-05	Calculate the readiness score and list missing documents.	System	High
FR-06	Search and filter scholarships by level, country, field and funding.	Public / Student	High
FR-07	Calculate a match score and classify Eligible / Potentially suitable / Not suitable.	System	High
FR-08	Request one of seven services and follow its status.	Student	High
FR-09	Track applications through eight stages with history.	Student / Advisor	High
FR-10	Review documents (approve / reject with a reason).	Advisor	High
FR-11	Assign requests, add notes and log calls / WhatsApp / emails.	Advisor	Medium
FR-12	Create and edit scholarships with full eligibility criteria.	Advisor / Admin	High
FR-13	Track admission results per scholarship.	Advisor	Medium
FR-14	Send notifications and deadline reminders.	System	Medium
FR-15	Manage users, roles and permissions.	Admin	High
FR-16	Show statistics: students, requests, applications, countries, services, acceptance, workload.	Admin	Medium
FR-17	Manage signed-in devices, email, password and notification preferences.	All	Medium
FR-18	Receive public “Contact us” messages.	Public	Low

2.3 Non-functional requirements

Table 7: *Non-functional requirements*

ID	Category	Requirement
NFR-01	Security	Passwords hashed with bcrypt; every SQL query uses prepared statements; role-based access on every route.
NFR-02	Security	Uploaded files stored outside the public folder with random names; MIME type checked from file content.
NFR-03	Usability	Full Arabic (RTL) and English (LTR) interface; responsive from 360 px phones to desktops.
NFR-04	Performance	API responses under 500 ms for normal queries; match results cached in `match_scores`.
NFR-05	Reliability	Daily database backup; transactions for multi-table writes.
NFR-06	Maintainability	Layered code (controllers, services, core); API documented with Swagger and Postman.
NFR-07	Ethics	AI output is decision support only and never claims guaranteed admission.

3. System Architecture

EduBridge follows a three-tier architecture. The React single-page application is the presentation layer. It talks to the PHP REST API over HTTPS using JSON. The API is the only component allowed to access the MySQL database and the file storage, and it calls external services (Groq API for language-model explanations, SMTP for email) through dedicated service classes.

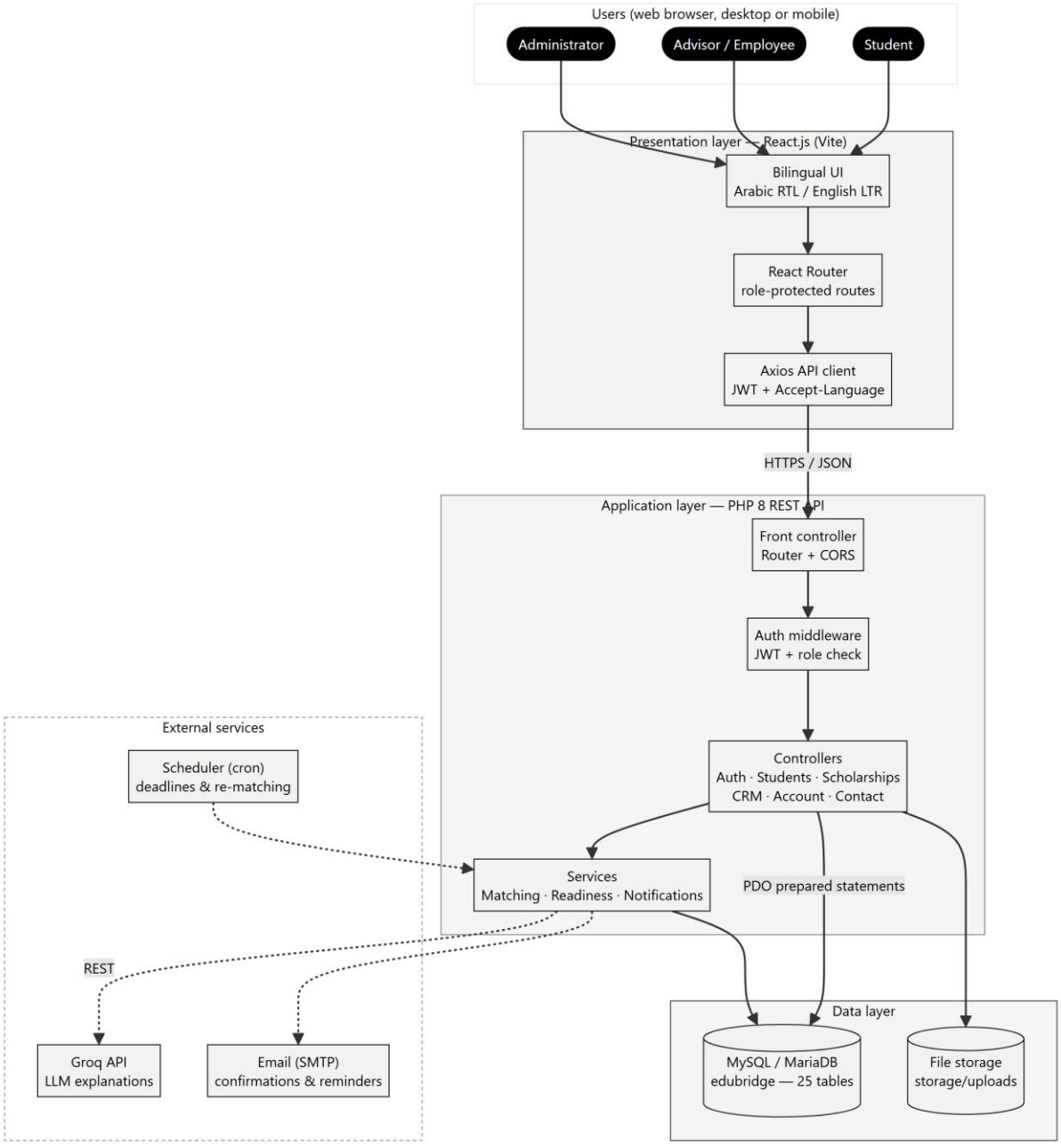


Figure 1: *Three-tier architecture of EduBridge*

Request flow

- The browser sends a request with `Authorization: Bearer <JWT>` and `Accept-Language` headers.
- The front controller (`public/index.php`) applies CORS and routes the request.
- The auth middleware checks the token, the session row and the role required by the route.

- The controller validates input, calls services and runs prepared SQL statements.
- A unified JSON response (or `{ "error": { … } }`) is returned.

4. Use Case Modeling

The use cases are split by actor to keep each diagram readable. Shared use cases such as signing in appear for every actor.

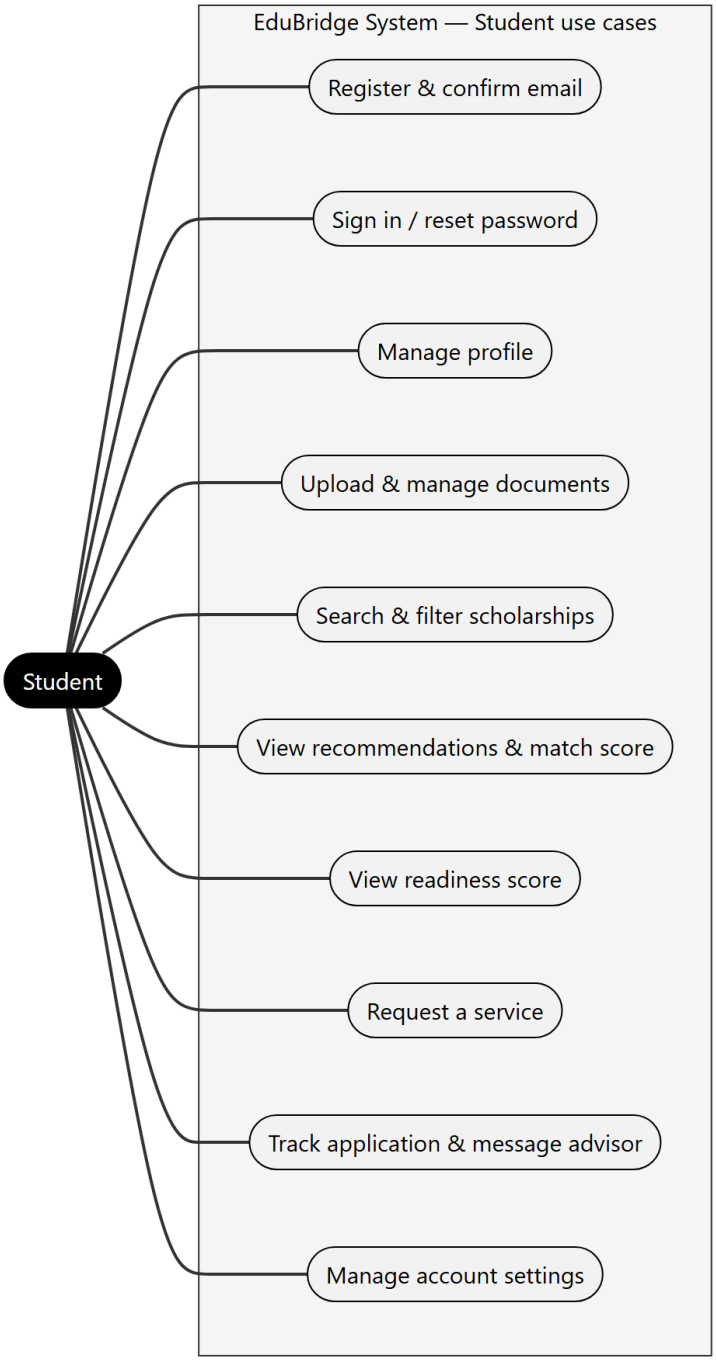


Figure 2: *Use case diagram — Student*

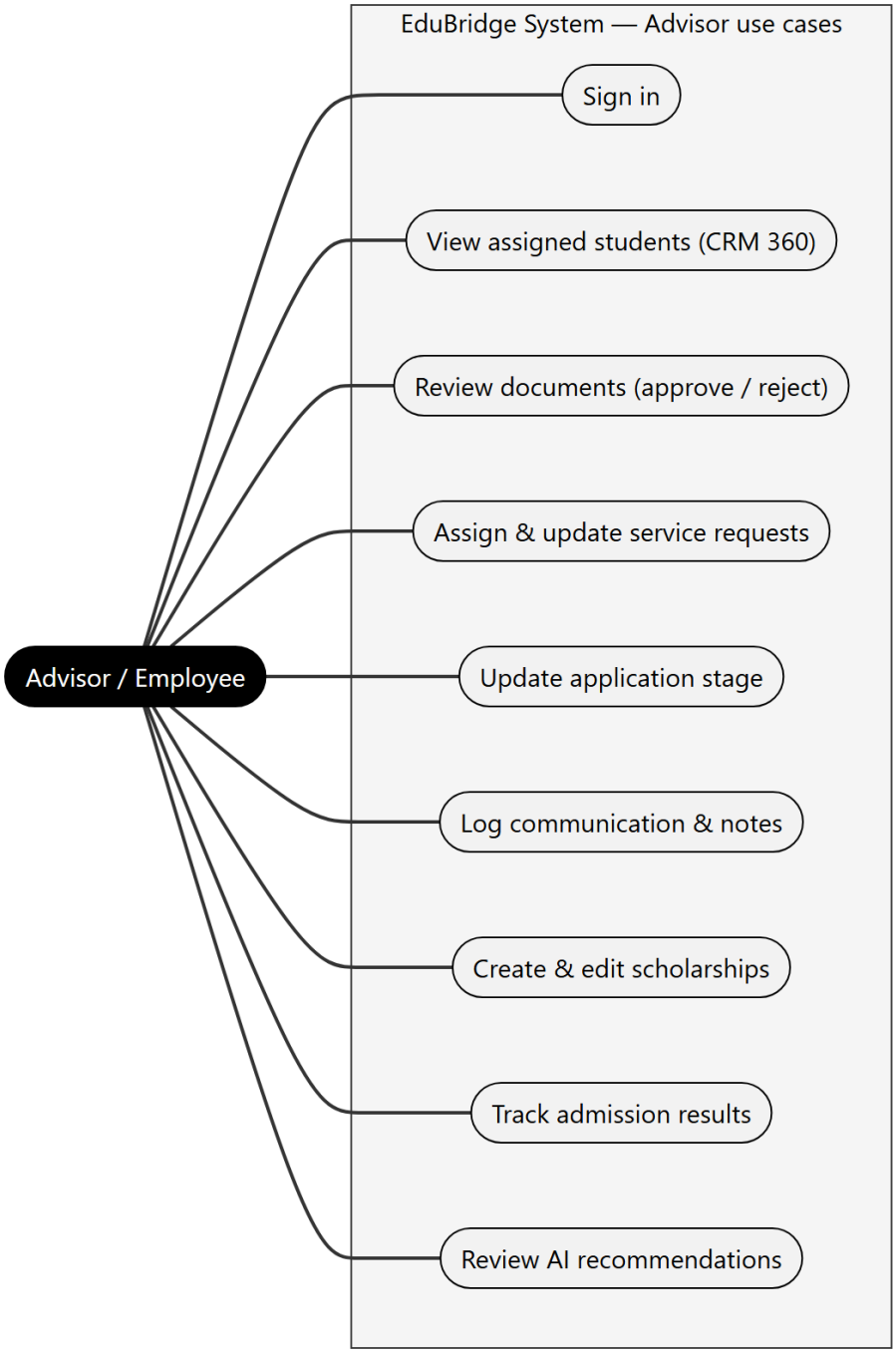


Figure 3: *Use case diagram — Advisor / Employee*

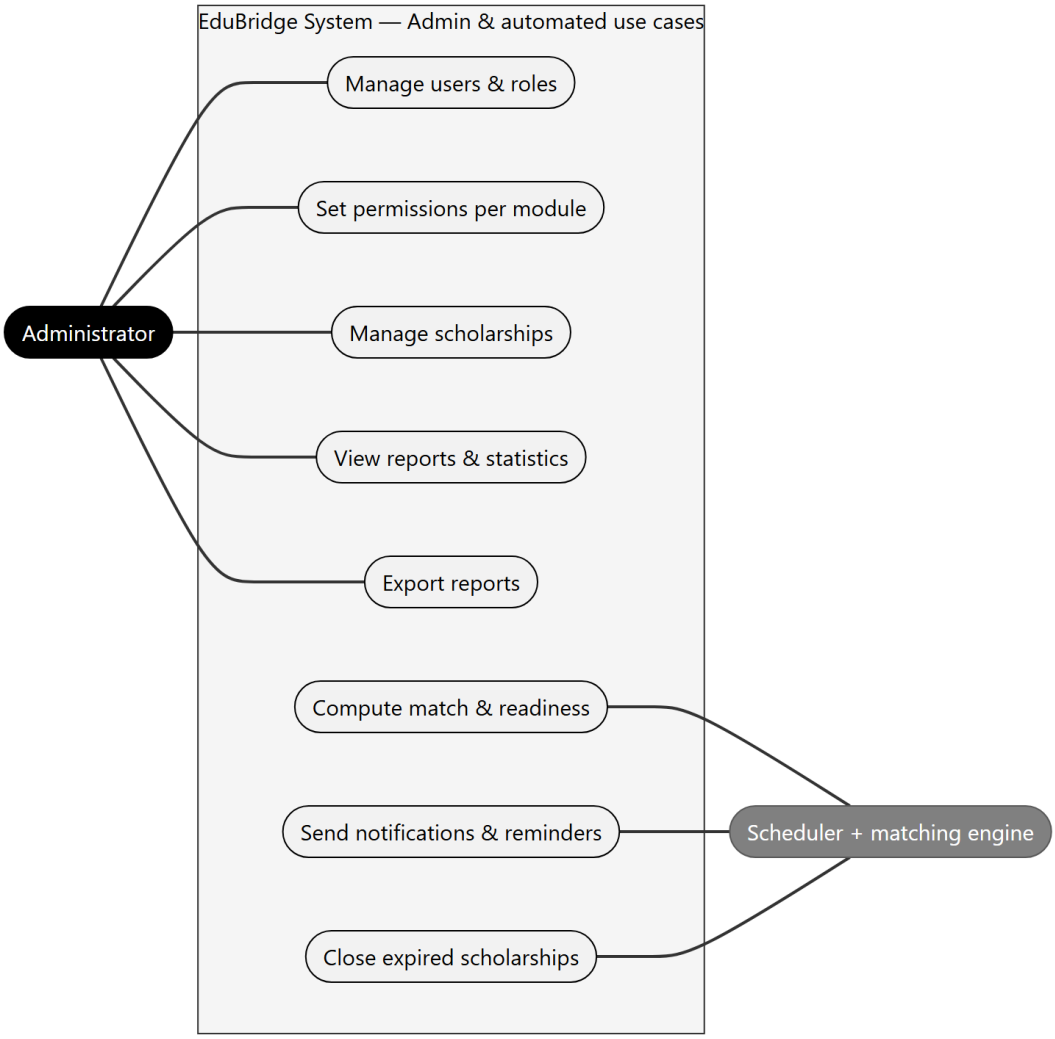


Figure 4: *Use case diagram — Administrator and automated system*

4.1 Use case descriptions

Table 8: *Use case UC-01*

Field	UC-01 Register a student account
Actor	Student (visitor)
Pre-condition	The email address is not registered.
Main flow	1. Visitor opens sign-up. 2. Enters account, personal, academic and goal data (5 steps). 3. Reviews and confirms. 4. System creates `users` and `students` rows in one transaction. 5. System sends a confirmation email and computes the first recommendations.
Alternative	Email already used → 409 `email_taken`; invalid data → 422 with the failing fields.
Post-condition	Student is signed in and sees the dashboard.

Table 9: *Use case UC-04*

Field	UC-04 Upload a document
Actor	Student
Pre-condition	Student is signed in.

Field	UC-04 Upload a document
Main flow	1. Student selects the document type and file. 2. System checks the size (≤ 10 MB) and the MIME type from the file content. 3. File is stored with a random name outside the public folder. 4. A `documents` row is created with status `underReview`. 5. Readiness score is recomputed.
Alternative	Unsupported type or too large → 422 `validation_failed`.
Post-condition	Advisor is notified to review the document.

Table 10: *Use case UC-13*

Field	UC-13 Update application stage
Actor	Advisor
Pre-condition	The application exists and is assigned to the advisor.
Main flow	1. Advisor opens Student 360. 2. Selects the next stage. 3. System updates `applications.stage` and inserts a row in `application_stage_history`. 4. Student is notified.
Alternative	Final stage → advisor records the result (accepted / rejected).
Post-condition	Timeline and reports are updated.

> **Traceability**
> Each use case maps to API endpoints (Section 10) and to database tables (Section 9), so every requirement can be followed from the interface to the data.

5. Class Diagrams

5.1 Domain model

The domain model shows the business objects and their relationships. `Student`, `Employee` and `Administrator` specialise `User` (generalisation). In the database this is implemented as one `users` table plus a `students` table in a one-to-one relationship, because only students have academic data.

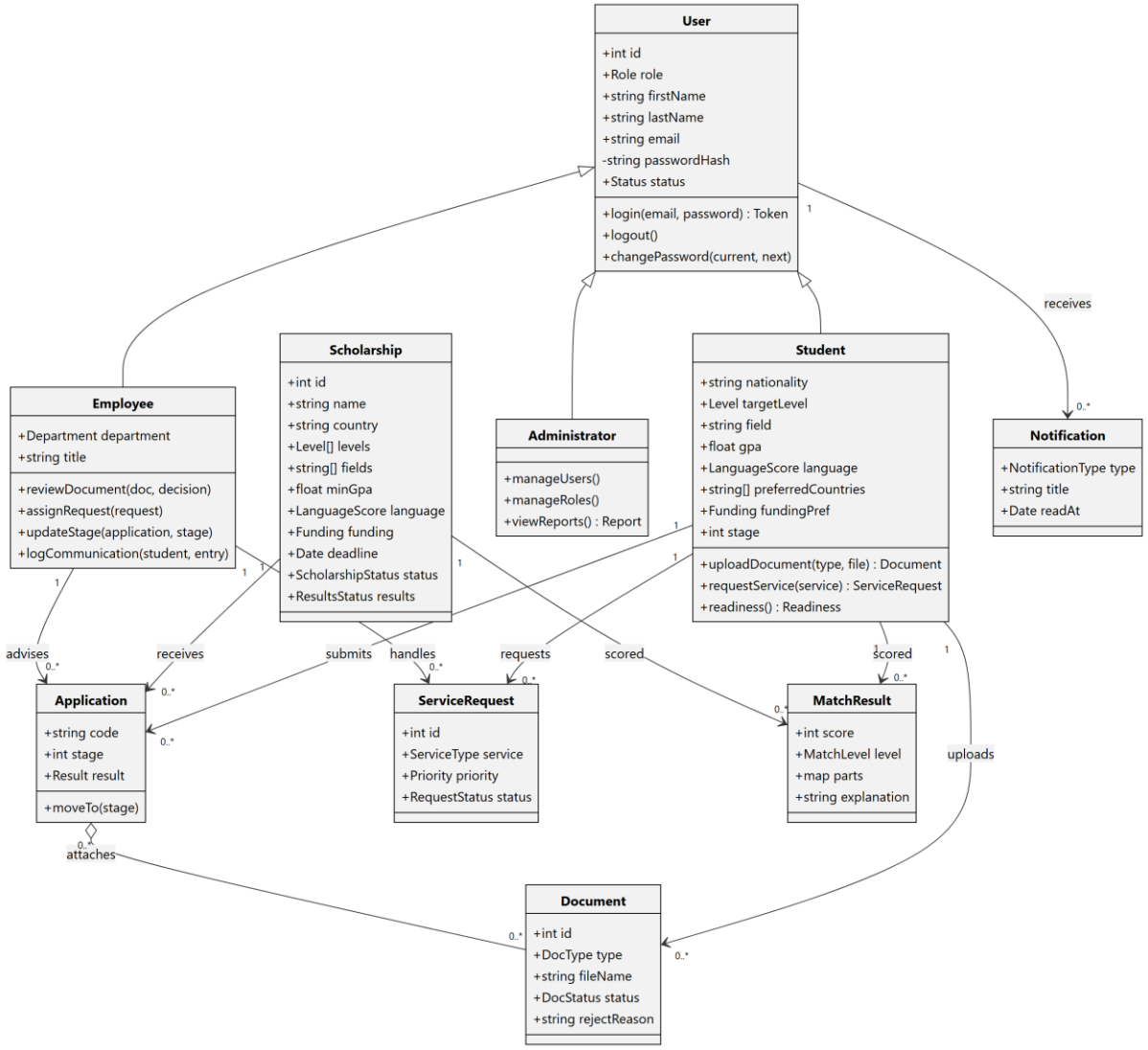


Figure 5: *Domain class diagram*

5.2 Back-end design classes

The PHP API is organised in three layers: **Core** (router, request/response, JWT, authentication, validation, database), **Services** (matching, mapping) and **Controllers** (one per resource). Static helpers keep the code small and easy to read for the team.

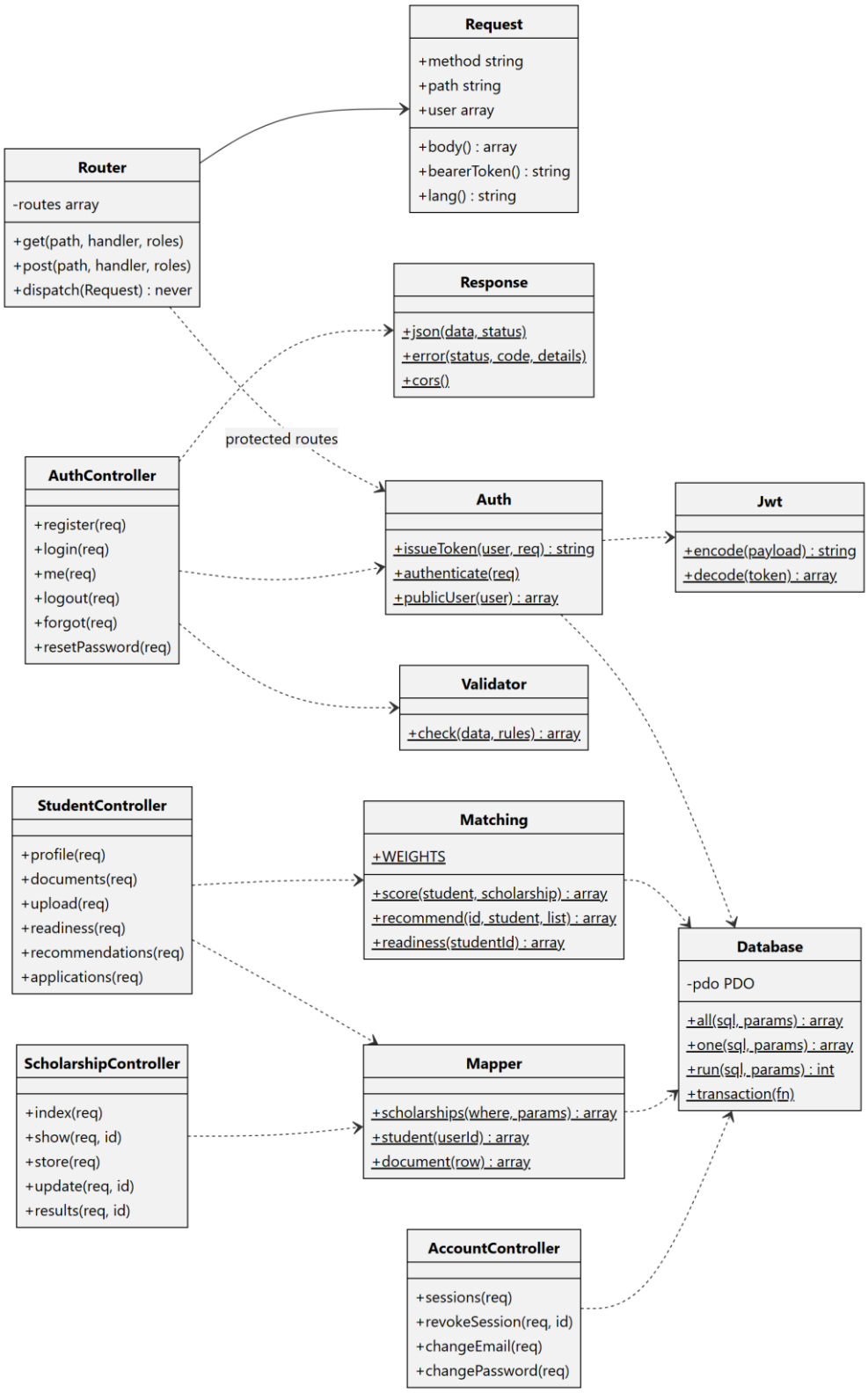


Figure 6: *Back-end class diagram (PHP API)*

Table 11: *Back-end classes*

Class	Responsibility
Router	Matches method + path, enforces the roles of protected routes, calls the controller.
Auth / Jwt	Issues signed tokens, stores one session per device, verifies and revokes tokens.
Database	Single PDO connection; `all`, `one`, `run` and `transaction` with prepared statements.

Class	Responsibility
Validator	Rule-based validation (required, email, min, max, numeric, between, in, date).
Matching	Weighted match score, classification and readiness score; caches results.
Mapper	Converts database rows (snake_case) to the JSON used by React (camelCase).

6. Sequence Diagrams

6.1 Sign in

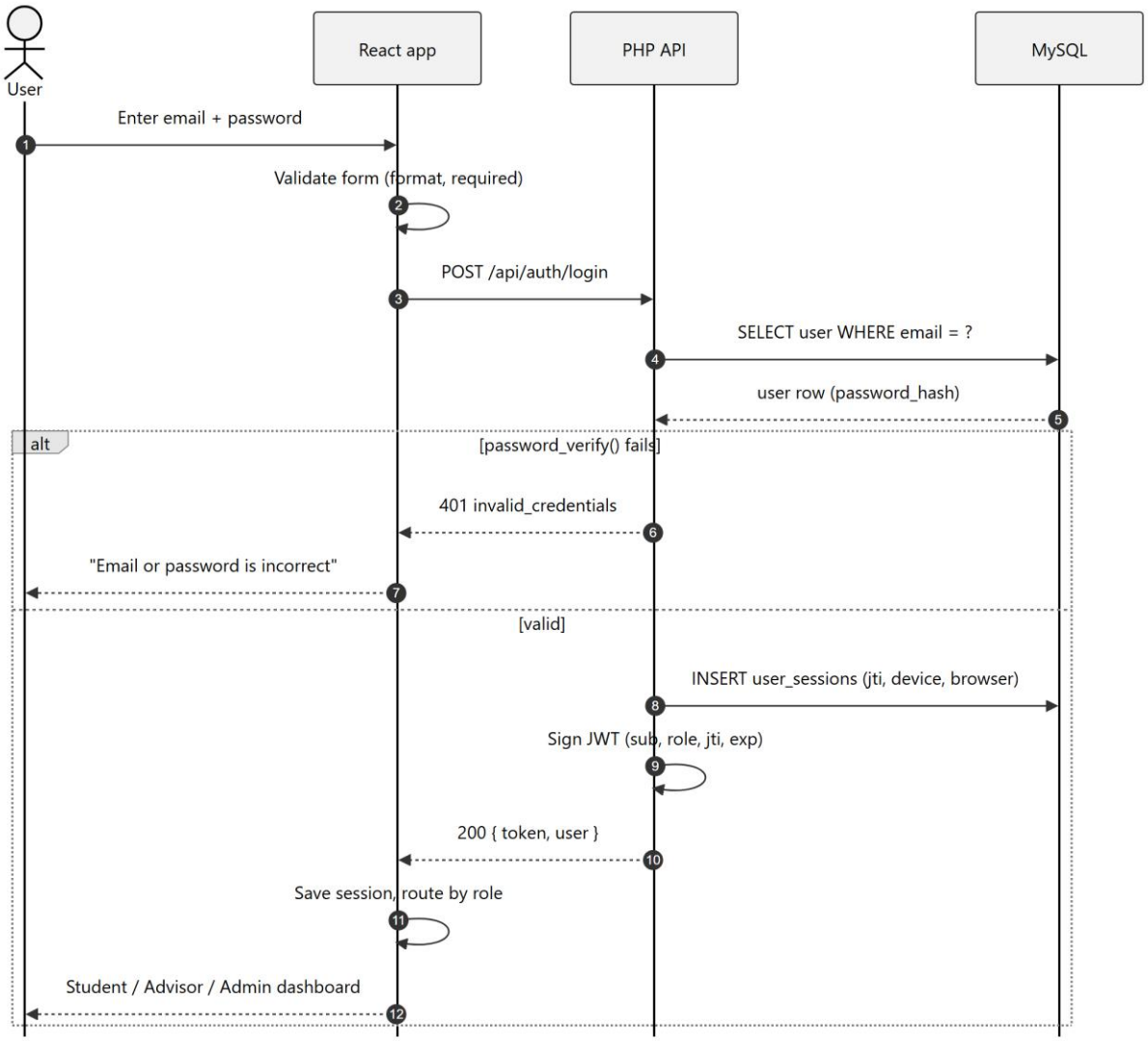


Figure 7: *Sequence diagram — sign in*

6.2 Upload a document and update readiness

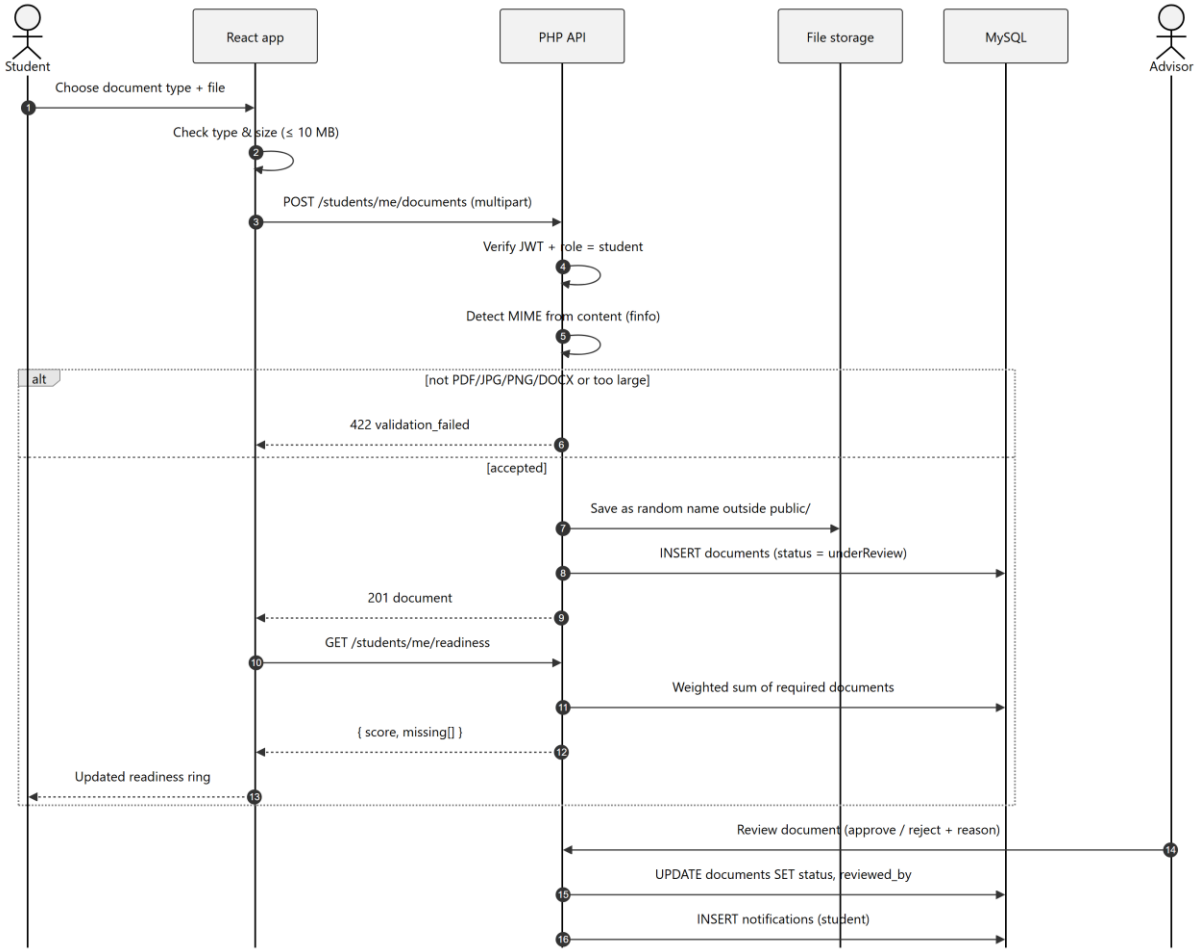


Figure 8: *Sequence diagram — document upload and review*

6.3 Service request handled by an advisor

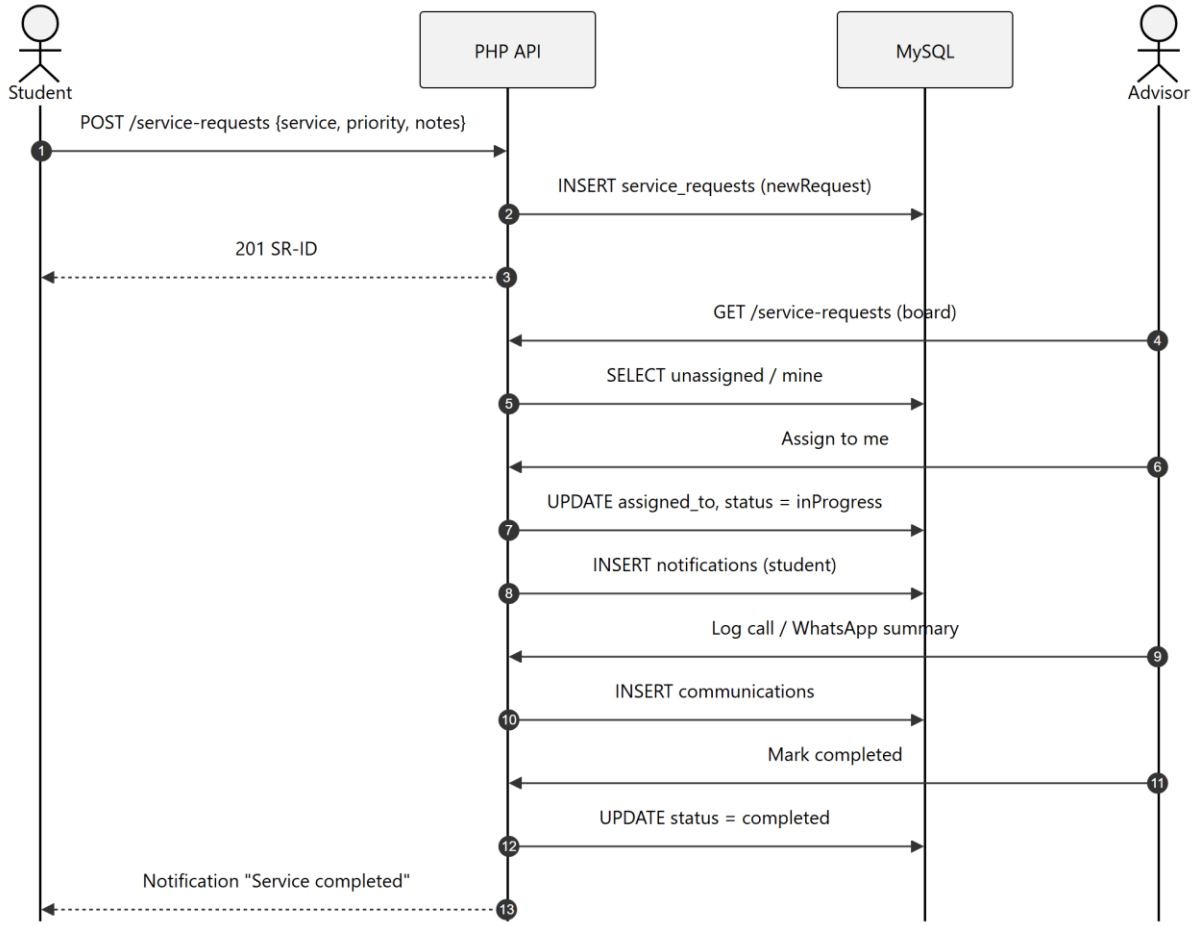


Figure 9: *Sequence diagram — service request*

7. Activity Diagrams

7.1 Student registration

Registration is split into five short steps to reduce errors. Each step is validated before moving on, and the account is created in one database transaction at the end.

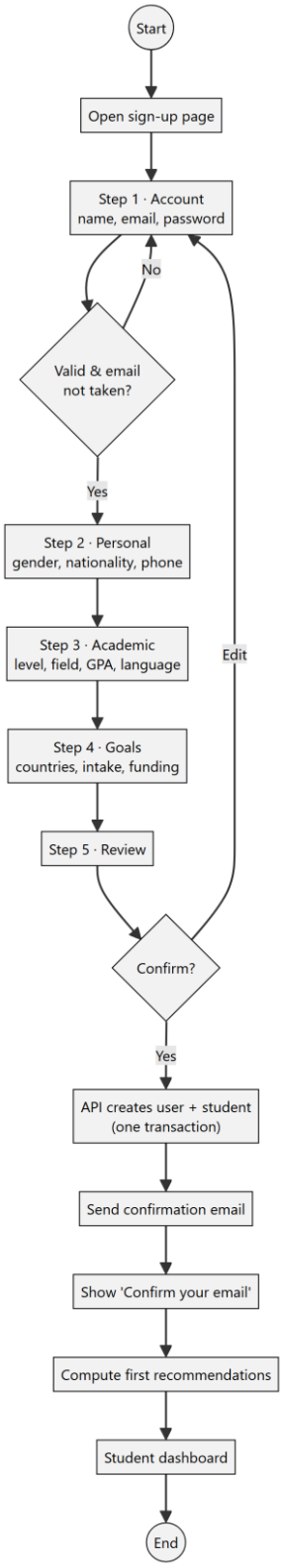


Figure 10: *Activity diagram — student registration*

7.2 Scholarship application journey

Each activity starts with the actor who performs it: Student, Advisor or System.

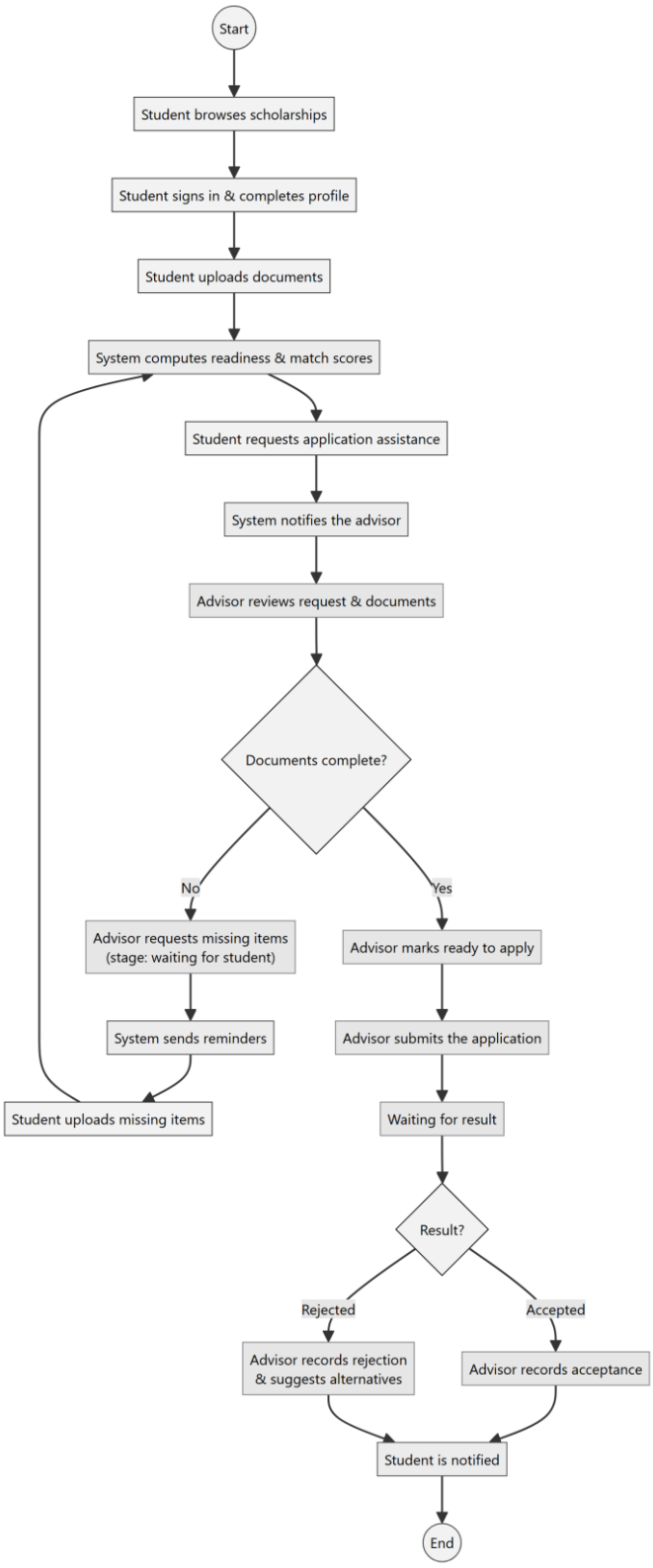


Figure 11: *Activity diagram — end-to-end application journey*

8. State Diagrams

State diagrams define the allowed status values stored in the database (ENUM columns) and the transitions the API accepts.

8.1 Application (8 stages)

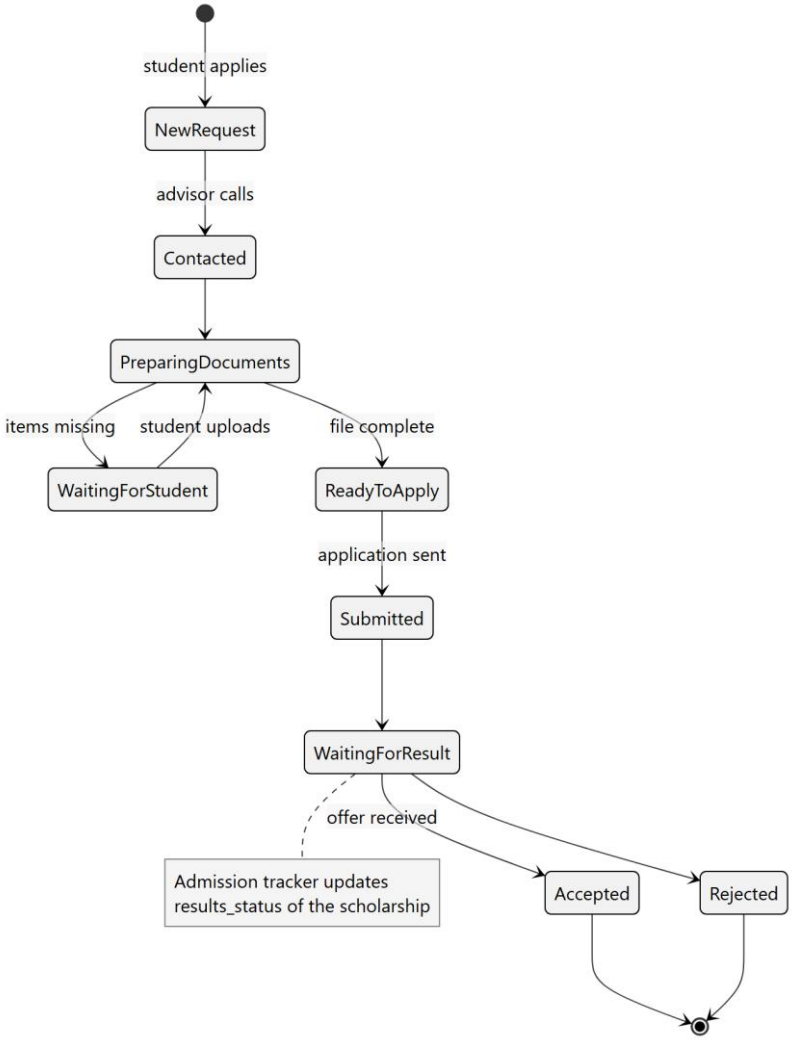


Figure 12: *State diagram — application*

Table 12: *Application stages*

Stage	Value in applications.stage	Meaning
New request	0	Student asked for help with this scholarship.
Contacted	1	Advisor made the first contact.
Preparing documents	2	File is being prepared.
Waiting for student	3	Something is missing from the student.
Ready to apply	4	File is complete.
Submitted	5	Application sent to the provider.
Waiting for result	6	Waiting for the decision.

Stage	Value in applications.stage	Meaning
Accepted / Rejected	7 + `result`	Final decision recorded.

8.2 Document

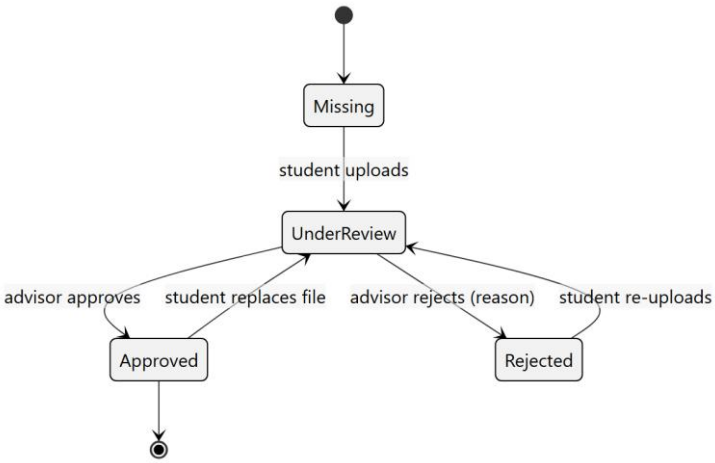


Figure 13: *State diagram — document review*

8.3 Service request

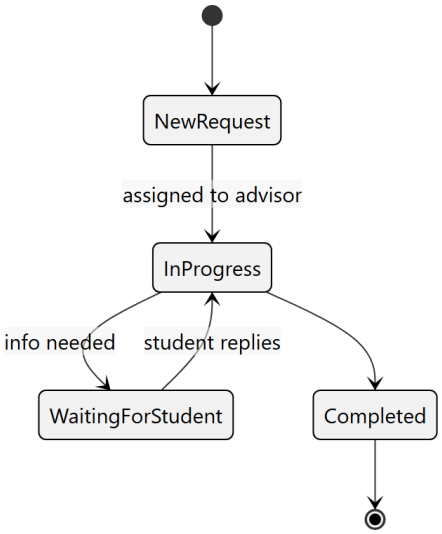


Figure 14: *State diagram — service request*

8.4 Scholarship and admission results

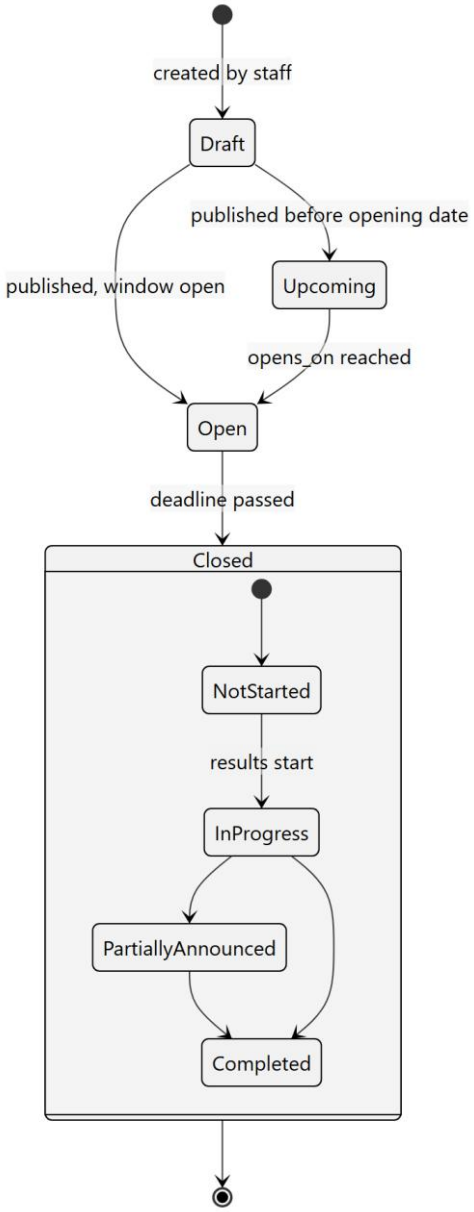


Figure 15: *State diagram — scholarship window and results*

8.5 User account

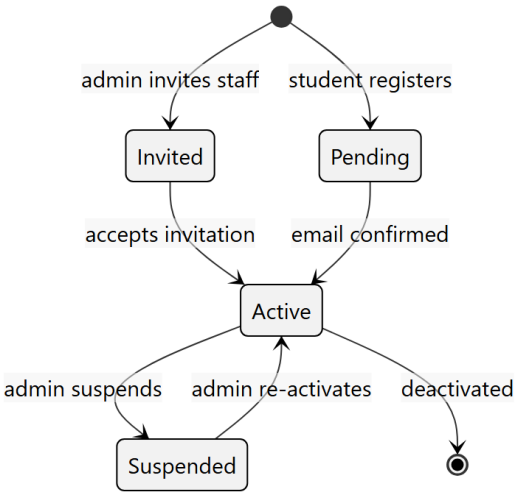


Figure 16: *State diagram — user account*

9. Database Design

9.1 Design approach

The database was designed top-down: entities were taken from the proposal modules, refined into a conceptual ERD, then into a logical relational model in **third normal form (3NF)**, and finally implemented in MySQL with InnoDB (transactions + foreign keys) and the `utf8mb4` character set so that Arabic and English text are stored correctly.

Table 13: *Main design decisions*

Decision	Reason
One `users` table for all roles + `students` (1 : 0..1)	Shared login and security for every role; only students have academic data.
Multi-valued attributes in their own tables	Levels, fields, nationalities and preferred countries need many values per row (1NF).
ENUM for fixed status values	The state diagrams define a closed set of values; ENUM rejects invalid data.
`match_scores` as a cache table	Scores are expensive to recompute on every page; they are refreshed when inputs change.
History tables (`application_stage_history`, `audit_log`)	Full traceability of who changed what and when.
Files on disk, metadata in MySQL	Keeps the database small; paths are not public.

9.2 Conceptual ERD

The overview shows the main entities and their relationships. Link tables are shown in the detailed ERDs that follow.

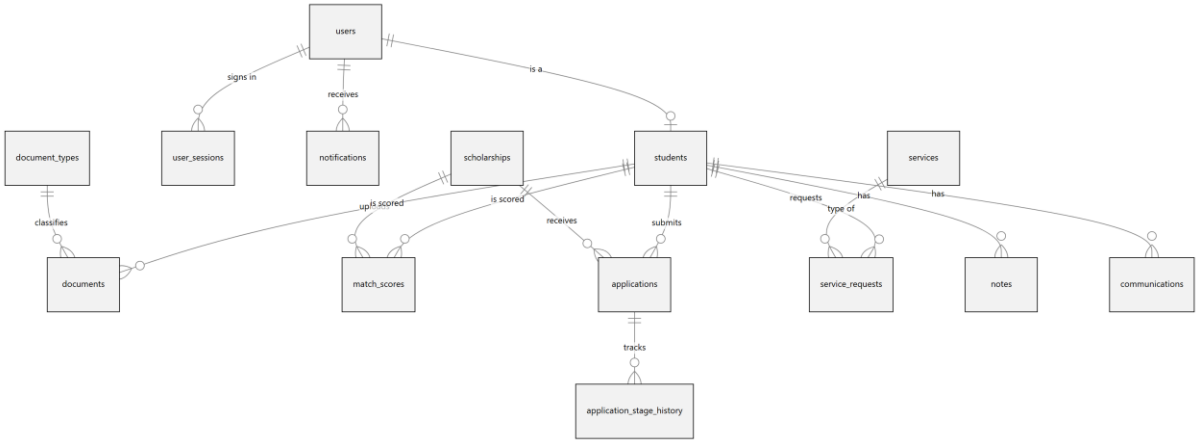


Figure 17: *Conceptual ERD (main entities)*

9.3 Logical ERDs by module

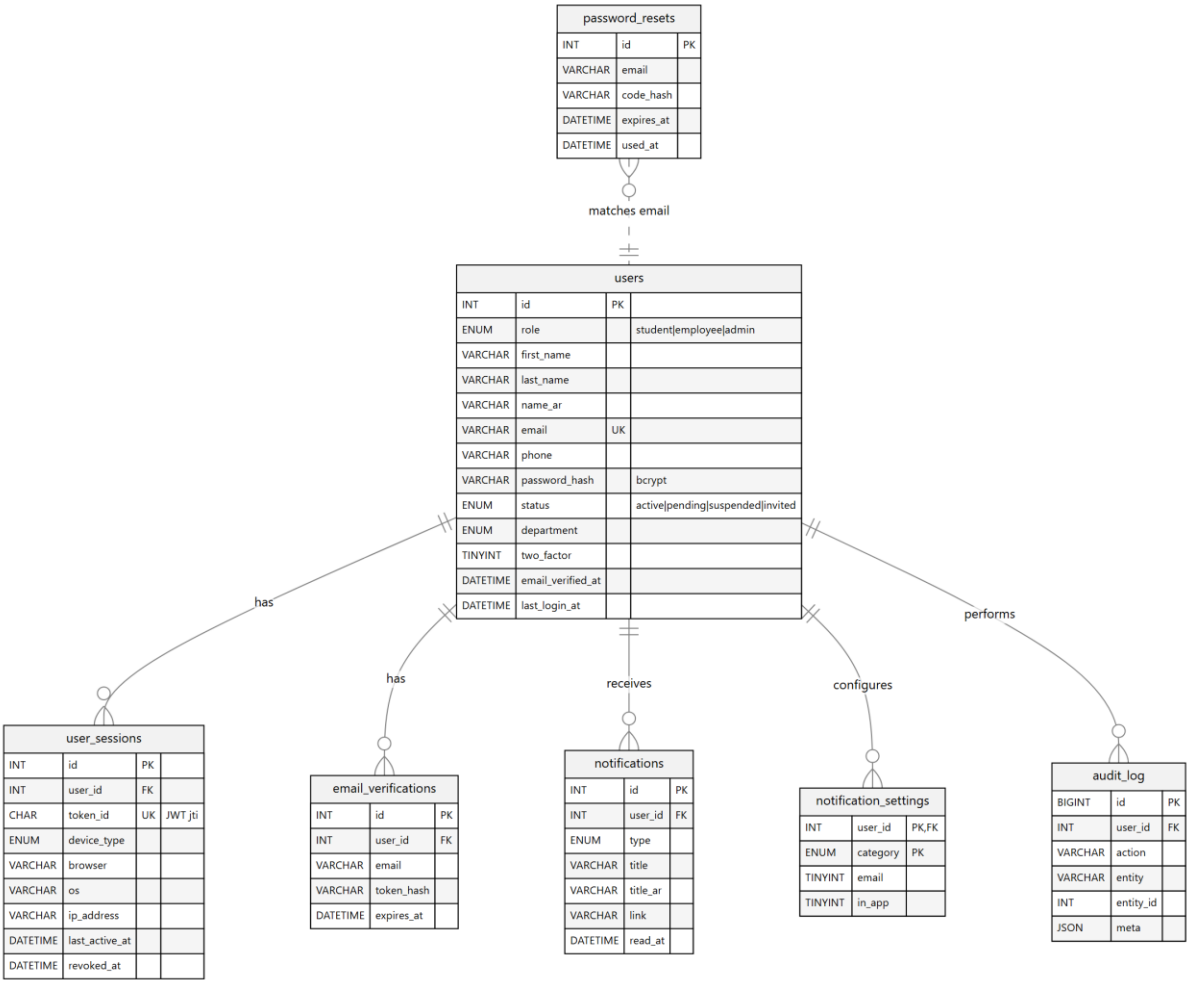


Figure 18: *ERD — users, authentication and notifications*

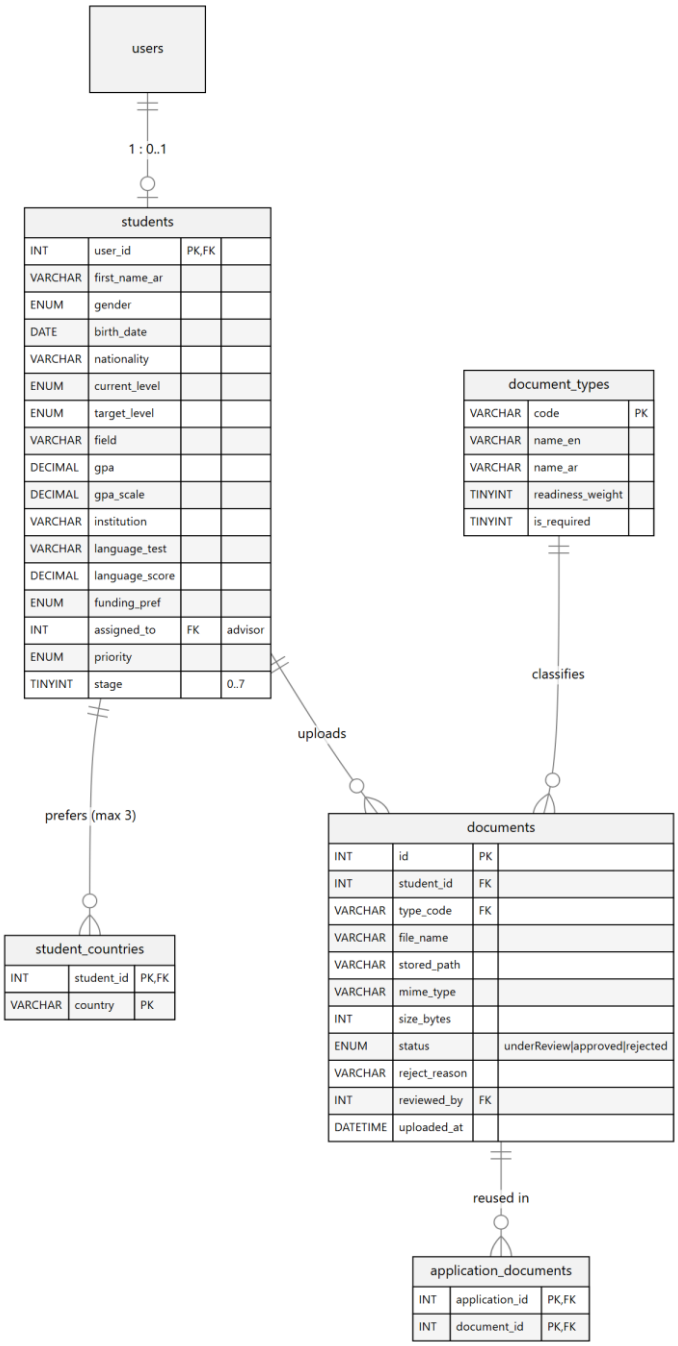


Figure 19: *ERD — students and documents*

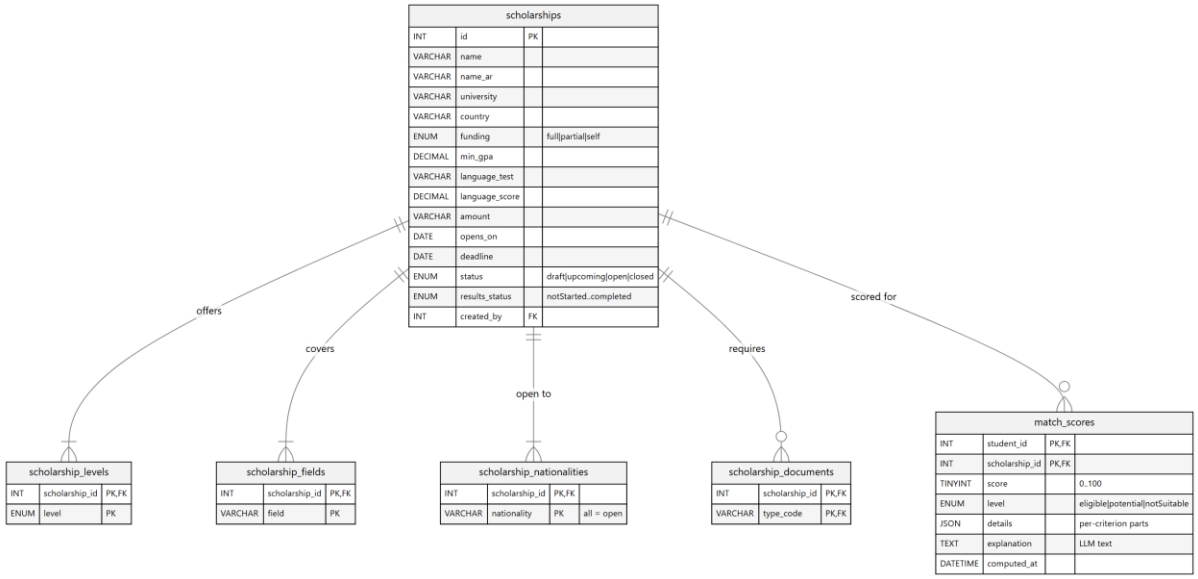


Figure 20: *ERD — scholarships and match results*

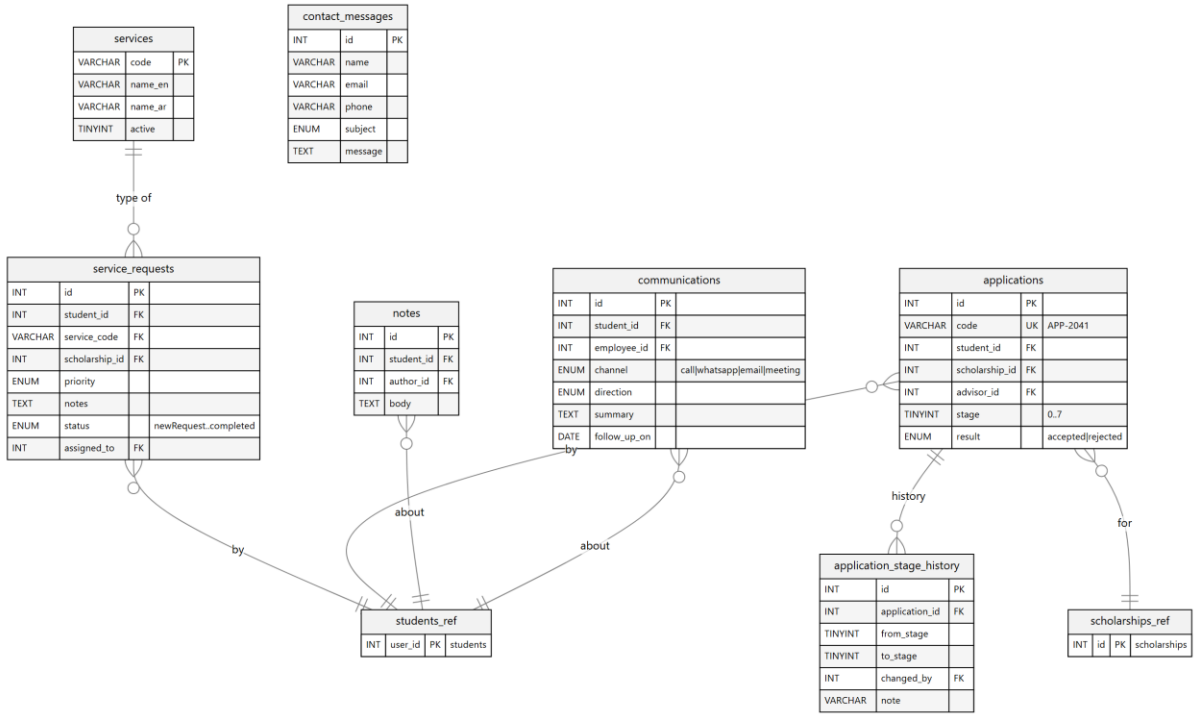


Figure 21: *ERD — services, applications and CRM*

9.4 Normalisation

- **1NF** — every column holds one atomic value. Repeating groups (study levels, fields, nationalities, preferred countries) were moved to their own tables, e.g. `scholarship_levels(scholarship_id, level)`.
- **2NF** — tables with composite keys (`student_countries`, `scholarship_fields`, `match_scores`, `notification_settings`) contain no attribute that depends on only part of the key.
- **3NF** — no transitive dependencies: document type names live in `document_types`, service names in `services`, and only codes are stored in `documents` and `service_requests`.
- **Controlled redundancy** — `match_scores` stores computed values on purpose (caching) and can be rebuilt at any time from the source tables.

9.5 Keys, constraints and indexes

Table 14: *Integrity rules*

Rule	Where	Purpose
Primary keys	Every table (AUTO_INCREMENT or composite)	Unique row identity
34 foreign keys	ON DELETE CASCADE for owned data, SET NULL for staff references	Referential integrity
UNIQUE	`users.email`, `user_sessions.token_id`, `applications.code`, (student, scholarship)	No duplicates
CHECK	`stage BETWEEN 0 AND 7`	Valid workflow stage
Indexes	`users(role,status)`, `scholarships(status,country,deadline)`, `documents(student_id,type_code)` …	Fast filtering and dashboards

9.6 Data dictionary

The dictionary below is generated directly from `database/schema.sql`, so it always matches the real database: **25 tables** and **199 columns**.

9.6.1 Users & authentication

users — Every account (student, advisor/employee, administrator) with credentials and status.

Column	Type	Null	Key	Default	Description
id	INT UNSIGNED	No	PK		Primary key (auto increment)
role	ENUM('student', 'employee', 'admin')	No		student	Access role
first_name	VARCHAR(80)	No			First name
last_name	VARCHAR(80)	No			Last name
name_ar	VARCHAR(160)	Yes			Full name in Arabic
email	VARCHAR(190)	No	UK		Email address (lower-case)
phone	VARCHAR(30)	Yes			Phone
password_hash	VARCHAR(255)	No			bcrypt hash of the password — never the plain password
status	ENUM('active', 'pending', 'suspended', 'invited')	No		active	Current status
department	ENUM('studentServices', 'admissions', 'visa', 'scholarships', 'compliance')	Yes			Staff department
title	VARCHAR(80)	Yes			Title
title_ar	VARCHAR(80)	Yes			Arabic version of title
avatar_path	VARCHAR(255)	Yes			Avatar path
two_factor	TINYINT(1)	No		0	2-step verification enabled
login_alerts	TINYINT(1)	No		1	Email on new device sign-in
email_verified_at	DATETIME	Yes			Email confirmation time
last_login_at	DATETIME	Yes			Last successful sign-in

Column	Type	Null	Key	Default	Description
created_at	DATETIME	No		CURRENT_TIMESTAMP	Creation time
updated_at	DATETIME	No		CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last update time (auto)

user_sessions — One row per signed-in device; the JWT id (jti) lets a device be signed out remotely.

Column	Type	Null	Key	Default	Description
id	INT UNSIGNED	No	PK		Primary key (auto increment)
user_id	INT UNSIGNED	No	FK		References users.id
token_id	CHAR(32)	No	UK		JWT id (jti) of the issued token
device_type	ENUM('laptop', 'phone', 'tablet')	No		laptop	laptop | phone | tablet
browser	VARCHAR(60)	Yes			Browser
os	VARCHAR(60)	Yes			Os
ip_address	VARCHAR(45)	Yes			IPv4/IPv6 address
city	VARCHAR(80)	Yes			City
created_at	DATETIME	No		CURRENT_TIMESTAMP	Creation time
last_active_at	DATETIME	No		CURRENT_TIMESTAMP	Last request from this device
revoked_at	DATETIME	Yes			Set when the device is signed out

password_resets — Hashed 6-digit reset codes with a 10-minute expiry.

Column	Type	Null	Key	Default	Description
id	INT UNSIGNED	No	PK		Primary key (auto increment)
email	VARCHAR(190)	No			Email address (lower-case)
code_hash	VARCHAR(255)	No			Code hash
expires_at	DATETIME	No			Expiry time
used_at	DATETIME	Yes			When the code/token was used
created_at	DATETIME	No		CURRENT_TIMESTAMP	Creation time

email_verifications — Pending email confirmations (sign-up and email change).

Column	Type	Null	Key	Default	Description
id	INT UNSIGNED	No	PK		Primary key (auto increment)
user_id	INT UNSIGNED	No	FK		References users.id
email	VARCHAR(190)	No			Email address (lower-case)
token_hash	VARCHAR(255)	No			Token hash
expires_at	DATETIME	No			Expiry time
used_at	DATETIME	Yes			When the code/token was used

Column	Type	Null	Key	Default	Description
created_at	DATETIME	No		CURRENT_TIMESTAMP	Creation time

9.6.2 Students & documents

students — Academic and personal profile of a student (1 : 0..1 with users).

Column	Type	Null	Key	Default	Description
user_id	INT UNSIGNED	No	PK, FK		References users.id
first_name_ar	VARCHAR(80)	Yes			Arabic version of first name
last_name_ar	VARCHAR(80)	Yes			Arabic version of last name
gender	ENUM('female', 'male')	Yes			Gender
birth_date	DATE	Yes			Birth date
nationality	VARCHAR(30)	Yes			Country code of nationality
country	VARCHAR(30)	Yes			Country code
city	VARCHAR(80)	Yes			City
current_level	ENUM('highSchool', 'bachelor', 'master')	Yes			Current level
target_level	ENUM('bachelor', 'master', 'phd')	Yes			Target level
field	VARCHAR(40)	Yes			Academic field code
gpa	DECIMAL(5,2)	Yes			Grade point average
gpa_scale	DECIMAL(5,2)	No		4.00	Maximum of the GPA scale (4 or 100)
institution	VARCHAR(160)	Yes			Institution
institution_ar	VARCHAR(160)	Yes			Arabic version of institution
graduation_year	SMALLINT	Yes			Graduation year
language_test	VARCHAR(20)	Yes			IELTS, TOEFL …
language_score	DECIMAL(5,1)	Yes			Test score (e.g. IELTS 6.5)
funding_pref	ENUM('full', 'partial', 'self')	Yes			Preferred funding type
intake	VARCHAR(40)	Yes			Intake
help_level	ENUM('full', 'partial', 'no')	Yes			Help level
assigned_to	INT UNSIGNED	Yes	FK		References users.id
priority	ENUM('high', 'medium', 'low')	No		medium	Handling priority
stage	TINYINT UNSIGNED	No		0	Workflow stage 0–7 (New request … Accepted/Rejected)
last_contact_at	DATETIME	Yes			Last contact at

student_countries — Preferred destination countries of a student (max. 3).

Column	Type	Null	Key	Default	Description
student_id	INT UNSIGNED	No	PK, FK		References students.user_id
country	VARCHAR(30)	No	PK		Country code

document_types — Catalogue of document types and their weight in the readiness score.

Column	Type	Null	Key	Default	Description
code	VARCHAR(30)	No	PK		Business code
name_en	VARCHAR(80)	No			English name
name_ar	VARCHAR(80)	No			Full name in Arabic
readiness_weight	TINYINT UNSIGNED	No		0	Weight in the readiness score (sum = 100)
is_required	TINYINT(1)	No		1	Counts in readiness
sort_order	TINYINT UNSIGNED	No		0	Display order

documents — Uploaded files, their review status and reviewer.

Column	Type	Null	Key	Default	Description
id	INT UNSIGNED	No	PK		Primary key (auto increment)
student_id	INT UNSIGNED	No	FK		References students.user_id
type_code	VARCHAR(30)	No	FK		References document_types.code
file_name	VARCHAR(190)	No			File name
stored_path	VARCHAR(255)	No			Path inside storage/uploads (not public)
mime_type	VARCHAR(80)	No			Detected MIME type
size_bytes	INT UNSIGNED	No			File size in bytes
status	ENUM('underReview', 'approved', 'rejected')	No		underReview	Current status
reject_reason	VARCHAR(255)	Yes			Why the advisor rejected the document
reviewed_by	INT UNSIGNED	Yes	FK		References users.id
reviewed_at	DATETIME	Yes			Review time
uploaded_at	DATETIME	No		CURRENT_TIMESTAMP	Uploaded at

9.6.3 Scholarships & matching

scholarships — Scholarship records with eligibility criteria, window and results status.

Column	Type	Null	Key	Default	Description
id	INT UNSIGNED	No	PK		Primary key (auto increment)
name	VARCHAR(190)	No			Name
name_ar	VARCHAR(190)	Yes			Full name in Arabic

Column	Type	Null	Key	Default	Description
university	VARCHAR(190)	No			University
university_ar	VARCHAR(190)	Yes			Arabic version of university
country	VARCHAR(30)	No			Country code
funding	ENUM('full', 'partial', 'self')	No		full	full | partial | self
min_gpa	DECIMAL(4,2)	Yes			Minimum GPA required
language_test	VARCHAR(20)	Yes			IELTS, TOEFL …
language_score	DECIMAL(5,1)	Yes			Test score (e.g. IELTS 6.5)
amount	VARCHAR(80)	Yes			Award value (text)
opens_on	DATE	Yes			Application window opens
deadline	DATE	Yes			Application deadline
status	ENUM('draft', 'upcoming', 'open', 'closed')	No		open	Current status
results_status	ENUM('notStarted', 'inProgress', 'partial', 'completed')	No		notStarted	Admission results progress
description	TEXT	Yes			English description
description_ar	TEXT	Yes			Arabic description
created_by	INT UNSIGNED	Yes	FK		References users.id
created_at	DATETIME	No		CURRENT_TIMESTAMP	Creation time
updated_at	DATETIME	No		CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last update time (auto)

scholarship_levels — Study levels a scholarship offers (multi-valued attribute).

Column	Type	Null	Key	Default	Description
scholarship_id	INT UNSIGNED	No	PK, FK		References scholarships.id
level	ENUM('bachelor', 'master', 'phd')	No	PK		Match level / study level

scholarship_fields — Academic fields a scholarship covers (multi-valued attribute).

Column	Type	Null	Key	Default	Description
scholarship_id	INT UNSIGNED	No	PK, FK		References scholarships.id
field	VARCHAR(40)	No	PK		Academic field code

scholarship_nationalities — Eligible nationalities; the value "all" means open to everyone.

Column	Type	Null	Key	Default	Description
scholarship_id	INT UNSIGNED	No	PK, FK		References scholarships.id
nationality	VARCHAR(30)	No	PK		Country code of nationality

scholarship_documents — Documents a scholarship requires.

Column	Type	Null	Key	Default	Description
scholarship_id	INT UNSIGNED	No	PK, FK		References scholarships.id
type_code	VARCHAR(30)	No	PK, FK		References document_types.code

match_scores — Cached result of the matching engine for each student–scholarship pair.

Column	Type	Null	Key	Default	Description
student_id	INT UNSIGNED	No	PK, FK		References students.user_id
scholarship_id	INT UNSIGNED	No	PK, FK		References scholarships.id
score	TINYINT UNSIGNED	No			Match score 0–100
level	ENUM('eligible', 'potential', 'notSuitable')	No			Match level / study level
details	JSON	Yes			Per-criterion partial scores (JSON)
explanation	TEXT	Yes			LLM explanation text (Groq)
computed_at	DATETIME	No		CURRENT_TIMESTAMP	When the match was computed

9.6.4 Services, applications & CRM

services — Catalogue of the seven consultancy services.

Column	Type	Null	Key	Default	Description
code	VARCHAR(40)	No	PK		Business code
name_en	VARCHAR(80)	No			English name
name_ar	VARCHAR(80)	No			Full name in Arabic
active	TINYINT(1)	No		1	Visible in the catalogue

service_requests — Service requests submitted by students and handled by advisors.

Column	Type	Null	Key	Default	Description
id	INT UNSIGNED	No	PK		Primary key (auto increment)
student_id	INT UNSIGNED	No	FK		References students.user_id
service_code	VARCHAR(40)	No	FK		References services.code
scholarship_id	INT UNSIGNED	Yes	FK		References scholarships.id
priority	ENUM('high', 'medium', 'low')	No		medium	Handling priority
notes	TEXT	Yes			Notes
status	ENUM('newRequest', 'inProgress', 'waitingStudent', 'completed')	No		newRequest	Current status
assigned_to	INT UNSIGNED	Yes	FK		References users.id
created_at	DATETIME	No		CURRENT_TIMESTAMP	Creation time

Column	Type	Null	Key	Default	Description
updated_at	DATETIME	No		CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last update time (auto)

applications — Scholarship applications and their position in the 8-stage workflow.

Column	Type	Null	Key	Default	Description
id	INT UNSIGNED	No	PK		Primary key (auto increment)
code	VARCHAR(20)	No	UK		Business code
student_id	INT UNSIGNED	No	FK, UK		References students.user_id
scholarship_id	INT UNSIGNED	No	FK, UK		References scholarships.id
advisor_id	INT UNSIGNED	Yes	FK		References users.id
stage	TINYINT UNSIGNED	No		0	Workflow stage 0–7 (New request … Accepted/Rejected)
result	ENUM('accepted', 'rejected')	Yes			accepted | rejected (final)
created_at	DATETIME	No		CURRENT_TIMESTAMP	Creation time
updated_at	DATETIME	No		CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last update time (auto)

application_documents — Documents reused in an application (many-to-many).

Column	Type	Null	Key	Default	Description
application_id	INT UNSIGNED	No	PK, FK		References applications.id
document_id	INT UNSIGNED	No	PK, FK		References documents.id

application_stage_history — Audit trail of every stage change of an application.

Column	Type	Null	Key	Default	Description
id	INT UNSIGNED	No	PK		Primary key (auto increment)
application_id	INT UNSIGNED	No	FK		References applications.id
from_stage	TINYINT UNSIGNED	Yes			Previous stage
to_stage	TINYINT UNSIGNED	No			New stage
changed_by	INT UNSIGNED	Yes	FK		References users.id
note	VARCHAR(255)	Yes			Short note
created_at	DATETIME	No		CURRENT_TIMESTAMP	Creation time

notes — Internal advisor notes about a student (CRM).

Column	Type	Null	Key	Default	Description
id	INT UNSIGNED	No	PK		Primary key (auto increment)
student_id	INT UNSIGNED	No	FK		References students.user_id

Column	Type	Null	Key	Default	Description
author_id	INT UNSIGNED	Yes	FK		References users.id
body	TEXT	No			Text
created_at	DATETIME	No		CURRENT_TIMESTAMP	Creation time

communications — Log of calls, WhatsApp messages, emails and meetings (CRM).

Column	Type	Null	Key	Default	Description
id	INT UNSIGNED	No	PK		Primary key (auto increment)
student_id	INT UNSIGNED	No	FK		References students.user_id
employee_id	INT UNSIGNED	Yes	FK		References users.id
channel	ENUM('call', 'whatsapp', 'email', 'meeting')	No			call | whatsapp | email | meeting
direction	ENUM('inbound', 'outbound')	No		outbound	inbound | outbound
summary	TEXT	No			Summary of the interaction
duration_min	SMALLINT UNSIGNED	Yes			Duration min
follow_up_on	DATE	Yes			Planned follow-up date
created_at	DATETIME	No		CURRENT_TIMESTAMP	Creation time

9.6.5 Notifications, contact & audit

notifications — In-app notifications per user (bilingual).

Column	Type	Null	Key	Default	Description
id	INT UNSIGNED	No	PK		Primary key (auto increment)
user_id	INT UNSIGNED	No	FK		References users.id
type	ENUM('deadline', 'document', 'service', 'scholarship', 'message', 'system')	No			Type
title	VARCHAR(190)	No			Title
title_ar	VARCHAR(190)	Yes			Arabic version of title
body	VARCHAR(500)	Yes			Text
body_ar	VARCHAR(500)	Yes			Arabic version of body
link	VARCHAR(255)	Yes			Front-end link
read_at	DATETIME	Yes			When the notification was read
created_at	DATETIME	No		CURRENT_TIMESTAMP	Creation time

notification_settings — Per-user notification preferences by category and channel.

Column	Type	Null	Key	Default	Description
user_id	INT UNSIGNED	No	PK, FK		References users.id

Column	Type	Null	Key	Default	Description
category	ENUM('requests', 'documents', 'deadlines', 'matches', 'messages')	No	PK		Notification category
email	TINYINT(1)	No		1	Email address (lower-case)
in_app	TINYINT(1)	No		1	In-app channel on/off

contact_messages — Messages from the public “Contact us” form.

Column	Type	Null	Key	Default	Description
id	INT UNSIGNED	No	PK		Primary key (auto increment)
name	VARCHAR(160)	No			Name
email	VARCHAR(190)	No			Email address (lower-case)
phone	VARCHAR(30)	No			Phone
subject	ENUM('general', 'scholarships', 'services', 'support')	No		general	Contact subject
message	TEXT	No			Message text
handled_at	DATETIME	Yes			When staff handled the message
created_at	DATETIME	No		CURRENT_TIMESTAMP	Creation time

audit_log — Security and activity log of important actions.

Column	Type	Null	Key	Default	Description
id	BIGINT UNSIGNED	No	PK		Primary key (auto increment)
user_id	INT UNSIGNED	Yes	FK		References users.id
action	VARCHAR(60)	No			Action
entity	VARCHAR(40)	Yes			Entity
entity_id	INT UNSIGNED	Yes			Entity id
meta	JSON	Yes			Extra data (JSON)
ip_address	VARCHAR(45)	Yes			IPv4/IPv6 address
created_at	DATETIME	No		CURRENT_TIMESTAMP	Creation time

9.7 SQL implementation

Table definitions (excerpt)

documents — file metadata with review status

```
CREATE TABLE documents (
  id           INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  student_id   INT UNSIGNED NOT NULL,
  type_code    VARCHAR(30)  NOT NULL,
  file_name    VARCHAR(190) NOT NULL,
  stored_path  VARCHAR(255) NOT NULL,
  mime_type    VARCHAR(80)  NOT NULL,
  size_bytes   INT UNSIGNED NOT NULL,
```

```
  status        ENUM('underReview','approved','rejected') NOT NULL DEFAULT 'underReview',
  reject_reason VARCHAR(255) NULL,
  reviewed_by   INT UNSIGNED NULL,
  reviewed_at   DATETIME     NULL,
  uploaded_at   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  KEY idx_documents_student (student_id, type_code),
  CONSTRAINT fk_documents_student  FOREIGN KEY (student_id)  REFERENCES students(user_id) ON DELETE
CASCADE,
  CONSTRAINT fk_documents_type     FOREIGN KEY (type_code)   REFERENCES document_types(code),
  CONSTRAINT fk_documents_reviewer FOREIGN KEY (reviewed_by) REFERENCES users(id) ON DELETE SET
NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

applications — 8-stage workflow

```
CREATE TABLE applications (
  id             INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  code           VARCHAR(20)  NOT NULL,
  student_id     INT UNSIGNED NOT NULL,
  scholarship_id INT UNSIGNED NOT NULL,
  advisor_id     INT UNSIGNED NULL,
  stage          TINYINT UNSIGNED NOT NULL DEFAULT 0,
  result         ENUM('accepted','rejected') NULL,
  created_at     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uq_applications_code (code),
  UNIQUE KEY uq_applications_pair (student_id, scholarship_id),
  CONSTRAINT fk_apps_student FOREIGN KEY (student_id)     REFERENCES students(user_id) ON DELETE
CASCADE,
  CONSTRAINT fk_apps_sch     FOREIGN KEY (scholarship_id) REFERENCES scholarships(id),
  CONSTRAINT fk_apps_advisor FOREIGN KEY (advisor_id)     REFERENCES users(id) ON DELETE SET NULL,
  CONSTRAINT chk_apps_stage CHECK (stage BETWEEN 0 AND 7)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Example queries used by the API

Prepared statements (the ? placeholders are bound by PDO)

```
-- Scholarship directory: master programmes in Germany (public page)
SELECT * FROM scholarships
 WHERE status <> 'draft' AND country = ?
   AND id IN (SELECT scholarship_id FROM scholarship_levels WHERE level = ?)
 ORDER BY FIELD(status,'open','upcoming','closed'), deadline;

-- Sign in: one session row per device (JWT id)
INSERT INTO user_sessions (user_id, token_id, device_type, browser, os, ip_address)
VALUES (?, ?, ?, ?, ?, ?);

-- Move an application to the next stage and keep the history (one transaction)
START TRANSACTION;
UPDATE applications SET stage = ? WHERE id = ?;
INSERT INTO application_stage_history (application_id, from_stage, to_stage, changed_by, note)
VALUES (?, ?, ?, ?, ?);
COMMIT;
```

Reporting views

Seven views implement the statistics required by the Reporting Module of the proposal. They are created by `database/views.sql`.

Table 15: *Reporting views*

View	Answers
v_totals	Number of students, requests, applications and decisions
v_countries_demand	Most requested destination countries
v_services_demand	Most requested services

View	Answers
v_application_stages	Application status statistics
v_acceptance_rate	Acceptance statistics
v_employee_workload	Employee workload
v_student_readiness	Readiness score of every student

v_employee_workload

```
CREATE OR REPLACE VIEW v_employee_workload AS
SELECT
  u.id, CONCAT(u.first_name, ' ', u.last_name) AS employee,
  (SELECT COUNT(*) FROM students s WHERE s.assigned_to = u.id)
AS students,
  (SELECT COUNT(*) FROM service_requests r WHERE r.assigned_to = u.id AND r.status <> 'completed')
AS open_requests,
  (SELECT COUNT(*) FROM service_requests r WHERE r.assigned_to = u.id AND r.status = 'completed')
AS completed_requests,
  (SELECT COUNT(*) FROM applications a WHERE a.advisor_id = u.id AND a.result IS NULL)
AS active_applications
FROM users u
WHERE u.role = 'employee';
```

Table 16: *Result of v_employee_workload on the demo data*

employee	students	open_requests	completed_requests	active_applications
Maya Khalil	4	1	0	2
Omar Farouk	1	0	1	0
Noor Hassan	1	0	0	0

9.8 Security

- Passwords and reset codes are stored only as **bcrypt hashes** (`password_hash`).
- All queries use **PDO prepared statements** with emulation disabled — no string concatenation of user input.
- Each token is linked to a `user_sessions` row; signing out or “sign out other devices” sets `revoked_at`.
- Routes declare their roles; the router returns **401** without a valid token and **403** for the wrong role.
- Uploaded files: MIME detected from content, 10 MB limit, random names, stored outside `public/`.
- The application database user should have only SELECT, INSERT, UPDATE, DELETE on `edubridge` in production.

9.9 Backup and recovery

Table 17: *Backup plan*

Task	How	When
Full backup	`mysqldump --single-transaction --routines edubridge > edubridge_YYYYMMDD.sql`	Daily, 02:00
Uploaded files	Copy `storage/uploads/` to backup storage	Daily
Retention	Keep 14 daily + 8 weekly copies	Automatic

Task	How	When
Restore test	Restore the latest dump into a test database and run the API health check	Monthly
Reset demo data	`php bin/setup.php --fresh`	Before presentations

10. API and Database Mapping

The table maps the implemented REST endpoints to the tables they read or write. The full interactive documentation is available at `http://localhost:8000/docs` (Swagger) and as a Postman collection.

Table 18: *Endpoints and tables*

Endpoint	Access	Tables
POST /api/auth/register	Public	users, students, student_countries, email_verifications, user_sessions
POST /api/auth/login · /logout	Public / signed in	users, user_sessions
POST /api/auth/forgot-password · /verify-code · /reset-password	Public	password_resets, users, user_sessions
GET /api/scholarships · /scholarships/{id}	Public	scholarships, scholarship_levels, scholarship_fields, scholarship_nationalities
POST · PUT /api/scholarships	Advisor / Admin	scholarships + link tables
GET · PUT /api/students/me	Student	users, students, student_countries
GET · POST /api/students/me/documents	Student	documents, document_types
GET /api/students/me/readiness	Student	documents, document_types
GET /api/students/me/recommendations	Student	students, scholarships…, match_scores
GET /api/students/me/applications	Student	applications, scholarships, users
POST /api/service-requests	Student	service_requests, services
/api/account/*	Signed in	user_sessions, users, email_verifications, notification_settings
POST /api/contact	Public	contact_messages

11. Conclusion

The analysis models and the database design together describe a complete, consistent system: every use case is supported by API endpoints, every endpoint uses well-defined tables, and every status value follows a state diagram. The database is implemented, seeded with demo data and verified through the running API, Swagger and an automated Postman run (33 requests passed).

> **Next steps**
> Implement the remaining CRM and administrator endpoints on the same tables.
> Add the scheduled jobs (deadline reminders, re-matching) described in the AI & Automation report.
> Prepare production configuration: restricted database user, HTTPS and automated backups.